

Systèmes, Paradigmes et Langages pour le Big Data

Sujet : Connected Component Finder

Yacine Abdelouhab

Sepanta Farzollahi

Paul Janin

Responsable : Dario Colazzo



1^{er} mars 2026

Abstract

Ce projet se concentre sur l'étude et l'implémentation de l'algorithme Connected Component Finder (CCF) pour l'analyse de graphes dans des environnements distribués. L'objectif principal est d'adapter un algorithme initialement conçu pour MapReduce à Apache Spark, en exploitant ses primitives de traitement parallèle et ses mécanismes de shuffle et de partitionnement.

Le projet explore plusieurs variantes de l'algorithme CCF, incluant des approches standard et optimisées, afin de gérer efficacement la propagation des identifiants minimaux à travers les composantes connexes des graphes. Chaque variante est analysée en termes de complexité algorithmique et de structure de traitement distribuée, en mettant l'accent sur la gestion de la mémoire et le parallélisme.

Les expérimentations sont réalisées sur des graphes de tailles et de topologies diverses, comprenant à la fois des graphes synthétiques et des graphes issus de réseaux réels. L'étude se concentre sur la mise en place d'un environnement expérimental Spark, la configuration du cluster local, et l'application systématique des phases d'itération et de déduplication sur l'ensemble des arêtes du graphe.

Table des matières

1	Introduction	1
1.1	Contexte et motivation	1
1.2	Problème étudié	1
1.3	Objectifs du projet	2
2	Méthodologie	3
2.1	Algorithme CCF	3
2.2	Principe des algorithmes et paradigme d'exécution	3
3	Conception des algorithmes	6
3.1	CCF-Dedup : nettoyage des redondances	6
3.2	Algorithme CCF-Iterate et ses variantes	7
3.2.1	Variante 1 : L'approche standard (vanilla)	7
3.2.2	Variante 2 : Version <i>RAM overload in disguise</i>	8
3.2.3	Variante 3 : Optimisation Spark par réduction et jointure	9
3.2.4	Variante 4 : Optimisation avancée par Secondary Sort natif	11
3.3	La boucle principale	13
4	Analyse expérimentale et discussions	15
4.1	Environnement expérimental	15
4.2	Jeux de données	16
4.2.1	Graphes synthétiques	16
4.2.2	Graphes réels	17
4.3	Résultats	17
4.4	Discussion	18
4.4.1	Analyse des performances et des architectures	19
4.4.2	Points forts et limites du cadre CCF	20
5	Conclusion	21
	Appendices	23
A	Code source	23

Chapitre 1

Introduction

L'étude des algorithmes distribués pour le traitement de graphes massifs illustre la puissance et les défis du calcul parallèle dans des contextes à grande échelle, offrant à la fois des perspectives théoriques et des applications concrètes.

1.1 Contexte et motivation

L'analyse de graphes à grande échelle est devenue un domaine central en informatique moderne, en particulier avec l'explosion des données issues des réseaux sociaux, du web, des systèmes biologiques et des infrastructures numériques. De nombreuses applications nécessitent l'identification de structures internes dans des graphes massifs contenant des millions, voire des milliards de sommets et d'arêtes.

Parmi les problèmes fondamentaux en analyse de graphes figure le calcul des *composantes connexes*. Une composante connexe est un sous-ensemble maximal de sommets tel que chaque paire de sommets est reliée par un chemin. Ce problème intervient dans de nombreux domaines, notamment l'analyse de réseaux sociaux, la détection de communautés, la recherche de clusters, et l'analyse d'infrastructures distribuées.

Cependant, lorsque la taille du graphe devient très importante, les algorithmes classiques séquentiels ne sont plus adaptés. Il devient alors nécessaire d'utiliser des modèles de calcul distribués tels que MapReduce afin de traiter efficacement de grands volumes de données.

1.2 Problème étudié

Ce projet s'appuie sur l'article "*CCF : Fast and Scalable Connected Component Computation in MapReduce*" ([Kardes et al. \(2014\)](#)), qui propose une méthode distribuée efficace pour le calcul des composantes connexes dans des graphes de grande taille.

L'algorithme proposé dans cet article repose sur une méthode de propagation de l'identifiant minimum (Min-ID) : à chaque composante du graphe est associé le plus petit identifiant du sommet qu'elle contient. Pour ce faire, CCF s'appuie sur deux opérations MapReduce ([Dean and Ghemawat \(2004\)](#)) principales :

- **CCF-Iterate**, qui explore le voisinage de chaque sommet, puis propage progressivement les identifiants minimaux au sein des composantes,

- **CCF-Dedup**, qui supprime les arêtes redondantes générées lors des itérations, pour réduire la charge réseau.

L'objectif principal est d'itérer ces deux étapes jusqu'à convergence, c'est-à-dire jusqu'à ce qu'aucune nouvelle propagation d'identifiant ne soit nécessaire. En d'autres termes, l'algorithme s'arrête lorsqu'aucune nouvelle paire n'est générée.

1.3 Objectifs du projet

Le travail réalisé dans ce projet consiste à :

- Comprendre en détail les deux algorithmes MapReduce proposés dans l'article
- Implémenter ces algorithmes en utilisant Apache Spark ([Zaharia et al. \(2016\)](#)) et son modèle de programmation basé sur les RDD (Resilient Distributed Datasets) ([Zaharia et al. \(2012\)](#))
- Étudier expérimentalement le comportement des algorithmes sur des graphes de tailles et topologies croissantes
- Analyser la **scalabilité** des implémentations face aux limites physiques que sont la RAM et le réseau
- Comparer les performances des variantes proposées : approche standard et approche optimisée (une adaptée à Spark, une qui simule le *secondary sorting* d'Hadoop)

L'adaptation à Spark permet de bénéficier d'un modèle distribué plus flexible que MapReduce, tout en conservant les principes fondamentaux de l'approche décrite dans l'article.

Chapitre 2

Méthodologie

La méthodologie adoptée pour l'implémentation et l'analyse expérimentale de l'algorithme CCF repose à la fois sur ses fondements théoriques et sur son adaptation au modèle d'exécution distribué de Spark.

2.1 Algorithme CCF

L'algorithme CCF (*Connected Component Finder*) a été conçu pour traiter des graphes massifs dans un environnement distribué. Son objectif est d'exploiter le calcul parallèle pour identifier efficacement les composantes connexes tout en minimisant les échanges de données entre les nœuds du cluster. Cette approche permet de gérer des graphes beaucoup plus volumineux que ce que pourraient traiter des algorithmes séquentiels classiques.

Le processus repose sur deux opérations fondamentales. La première, **CCF-Iterate** (Section 3.2), utilise la stratégie du **Min-ID** : elle propage l'identifiant minimal au sein de chaque composante connexe. À chaque itération, un nœud explore son voisinage et adopte le plus petit identifiant observé. La seconde, **CCF-Dedup** (Section 3.1), supprime les paires redondantes générées au fil des itérations afin de limiter le volume de données échangées sur le réseau et d'alléger le coût des opérations de *shuffle*. Si **CCF-Iterate** produit au moins une nouvelle paire lors d'une itération, le processus est relancé depuis le début. L'algorithme converge et s'arrête lorsque plus aucune nouvelle paire n'est générée.

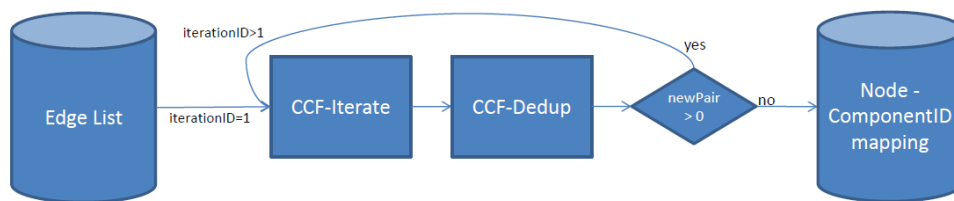


Figure 2.1 : Processus itératif de l'algorithme CCF

2.2 Principe des algorithmes et paradigme d'exécution

L'article de référence s'appuie sur le modèle **MapReduce**, qui constitue le socle théorique de la conception des algorithmes CCF. Il s'organise en trois étapes successives qu'il est essentiel

de bien comprendre pour saisir les enjeux d'optimisation.

Phase Map

Les données sont d'abord distribuées sur plusieurs partitions. La fonction **Map** est appliquée indépendamment sur chaque élément (ici, les arêtes du graphe) et produit des paires clé-valeur intermédiaires. Cette étape est locale et massivement parallélisée.

Phase Shuffle and Sort

Les paires intermédiaires sont redistribuées vers les *reducers* selon leur clé. Une fonction de hachage fixe la destination de chaque clé, de sorte que toutes les occurrences d'une même clé aboutissent sur la même machine. Dans le modèle MapReduce classique, le framework trie nativement les données sur le disque lors de ce transfert. À l'issue de cette étape, chaque *reducer* dispose de l'ensemble des valeurs associées à une clé donnée. Cette phase est la plus coûteuse du processus, car elle implique des échanges conséquents sur le réseau.

Phase Reduce

Chaque *reducer* traite une clé k et les valeurs associées $(k, [v_1, \dots, v_n])$. La fonction **Reduce** applique alors l'opération d'agrégation définie par l'algorithme pour produire le résultat final correspondant à la clé.

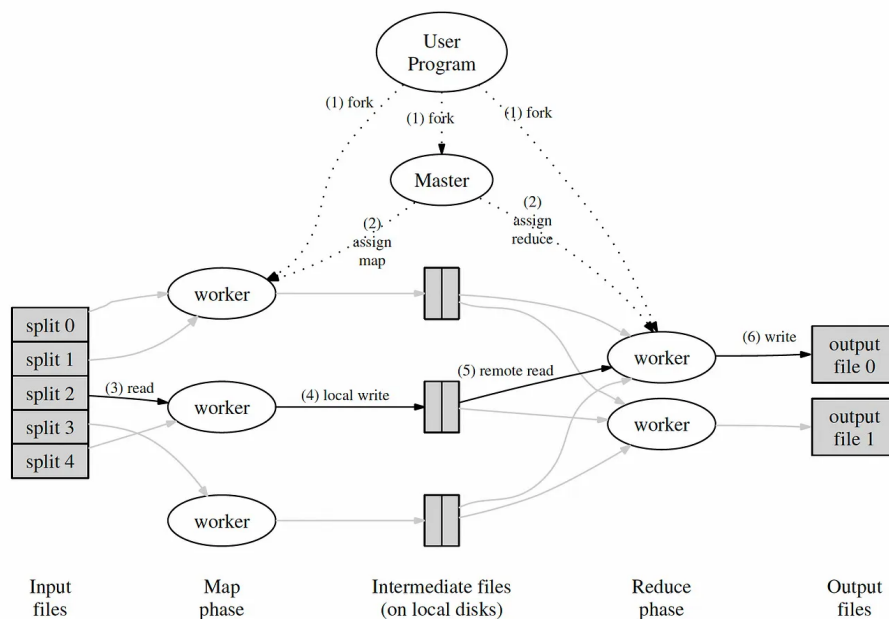


Figure 2.2 : Architecture d'exécution d'un job MapReduce

Passage du paradigme MapReduce à Spark

Bien que la logique mathématique des algorithmes CCF soit pensée pour MapReduce, **l'implémentation réalisée dans ce projet s'appuie sur le framework Apache Spark** et son modèle RDD.

Cette transition introduit des différences architecturales majeures qui justifieront nos choix d'implémentation par la suite :

- **Stockage en RAM vs disque** : Hadoop MapReduce écrit les résultats intermédiaires sur disque entre chaque itération, ce qui crée une forte latence d'initialisation. Spark, au contraire, privilégie le maintien des données en mémoire vive, accélérant considérablement les algorithmes itératifs comme CCF.
- **Lazy evaluation** : Spark gère deux types d'opération : les transformations et les actions. Les transformations (par exemple `flatMap`, `join` ou `reduceByKey`) n'appellent aucun calcul immédiat, mais incitent Spark à construire un plan d'exécution (sous forme d'un digraphe acyclique). Pour que les transformations soient effectivement appliquées, la lazy evaluation a besoin d'une action (`collect`, `count`, *etc.*). Cela nécessitera une gestion explicite de la lignée des données (via *checkpointing*) pour éviter une surcharge du moteur lors des itérations successives.
- **Gestion du tri** : Contrairement à Hadoop¹, Spark ne trie pas nativement les valeurs lors du *shuffle*. Les optimisations telles que le *secondary sort* devront donc être repensées en utilisant des fonctions de réduction distribuées propres à Spark.

Ainsi, la solution adoptée dans ce projet consiste à implémenter l'algorithme CCF en PySpark sous quatre variantes distinctes : une traduction littérale de l'algorithme 1 du papier (qui utilise `groupByKey`), une approche naïve de l'algorithme optimisé (naïve car tri en mémoire) et enfin, deux versions différentes de la véritable optimisation pour Spark (`reduceByKey` et `jointure / repartitionAndSortWithinPartitions`). Cette approche comparative nous permettra d'illustrer concrètement l'impact des choix d'architecture logicielle sur les performances (RAM et réseau, notamment).

1. Hadoop écrit les données sur le disque dur et trie les clés. Le *secondary sort* est une optimisation quasi-native, qui consiste à trier en plus les valeurs, sans réel surcoût.

Chapitre 3

Conception des algorithmes

Après avoir présenté les principes théoriques de l'algorithme CCF et son cadre d'exécution distribué, il convient désormais de détailler les choix de conception retenus pour son implémentation sous Spark. L'objectif est d'analyser concrètement l'impact des différentes stratégies de regroupement, de tri et de propagation sur la consommation mémoire, les communications réseau et les performances globales.

3.1 CCF-Dedup : nettoyage des redondances

L'algorithme CCF-Dedup, commun à toutes les versions de CCF, a pour rôle d'éliminer les paires en double générées lors de l'itération précédente. Spark propose une fonction native `distinct()` qui permet d'obtenir le même résultat. Toutefois, nous avons implémenté la logique originale de l'article pour illustrer le fonctionnement du *shuffle & sort*.

Phase Map

Chaque paire (u, v) est transformée en une paire clé-valeur dont la clé est la paire elle-même :

$$(u, v) \mapsto ((u, v), \text{None}).$$

Deux occurrences identiques de l'arête produisent donc exactement la même clé.

Phase Reduce

On applique ensuite une réduction par clé via `reduceByKey`. La fonction de réduction prend deux valeurs associées à une même clé et retourne simplement la première (v_1). Puisque toutes les valeurs instanciées lors du Map valent `None`, cette opération combine les occurrences identiques en flux (Map-side combine) et ne conserve qu'une seule instance de la clé après le *shuffle*. Contrairement à un `groupByKey`, aucune liste n'est matérialisée. Enfin, on supprime la valeur `None` pour reconstruire la paire originale unique.

Algorithm 1 CCF-Dedup (une itération)**Input** : RDD des paires (u, v) **Output** : RDD des paires sans redondance

```

1 def ccf_dedup(rdd):
2     mapped_rdd = rdd.map(lambda edge: (edge, None))
3
4     # Shuffle & Sort va regrouper toutes les keys identiques
5     # On utilise reduceByKey pour ne garder qu'une seule occurrence.
6     reduced_rdd = mapped_rdd.reduceByKey(lambda v1, v2: v1)
7     final_rdd = reduced_rdd.map(lambda kv: kv[0])
8
9     # On pourra comparer cette implementation avec le distinct() natif en
10    Spark par la suite
11
12    return final_rdd

```

3.2 Algorithme CCF-Iterate et ses variantes

L'algorithme CCF-Iterate constitue le cœur du processus de propagation des identifiants minimaux dans le graphe. Cette section présente successivement plusieurs variantes de son implémentation, de la version naïve en mémoire à des optimisations adaptées à Spark, en illustrant les compromis entre consommation mémoire, complexité computationnelle et volume des échanges réseau.

3.2.1 Variante 1 : L'approche standard (vanilla)

Cette première implémentation suit strictement l'Algorithme 1 décrit dans l'article et repose sur une stratégie de regroupement des données (dans la mémoire vive!).

Phase Map

À partir de chaque paire (u, v) représentant une arête du graphe, nous émettons deux paires directionnelles : (u, v) et (v, u) grâce à la fonction `flatMap`. Cette duplication garantit que l'information circulera dans les deux sens (de fait, on se trouve dans le cas non-orienté) et que chaque nœud recevra l'ensemble de ses voisins lors de la phase de regroupement.

Phase Reduce

Les paires intermédiaires sont regroupées par clé à l'aide de la transformation `groupByKey`. Cette opération force toutes les valeurs associées à une clé u à être rassemblées dans une même tâche. Le *reducer* dispose alors simultanément de l'itérable de toutes les valeurs associées à u . On convertit cet itérable en liste explicite en RAM pour calculer :

$$m = \min(u, v_1, \dots, v_n).$$

Algorithm 2 CCF-Iterate (une itération)**Input** : RDD des arêtes (u, v) , accumulateur Spark (variable globale)**Output** : RDD des nouvelles paires générées

```

1 def ccf_iterate_vanilla(rdd, new_pair_accum):
2
3     bothDirections = rdd.flatMap(lambda x: [(x[0], x[1]), (x[1], x[0])])
4
5     def process_vanilla(key_values):
6         key, values = key_values
7         val_list = list(values)
8         min_val = min(val_list)
9
10        emitted = []
11        if min_val < key:
12            emitted.append((key, min_val))
13            for v in val_list:
14                if v != min_val:
15                    new_pair_accum.add(1)
16                    emitted.append((v, min_val))
17        return emitted
18
19    return bothDirections.groupByKey().flatMap(process_vanilla)

```

Note : L'itérable PySpark renvoyé par le `groupByKey` ne permet de parcourir les données qu'une seule fois sans tout charger dans la RAM. Or, ici le *reducer* doit trouver le minimum de la liste, et émettre les paires ce qui nécessite 2 passes sur les données. On doit donc charger l'intégralité des valeurs en RAM ce qui est fait automatiquement par la conversion en liste `list(values)`.

Si $m = u$, aucune nouvelle paire n'est émise. Si $m < u$, on produit la paire (u, m) . Puis, on itère sur la liste pour lier chaque voisin $v_i \neq m$ au minimum m .

L'utilisation de `groupByKey` simule le regroupement complet des voisins. Cependant, la conversion en liste impose une complexité spatiale en $\mathcal{O}(N)$ (où N est le degré du nœud). Sur des nœuds très denses (hubs), cette approche sature la RAM du *worker* et provoque des erreurs de type *Out Of Memory*.

3.2.2 Variante 2 : Version RAM overload in disguise

En cherchant quelques détails supplémentaires sur les subtilités du papier CCF, nous sommes tombés sur une implémentation erronée¹ de la version optimisée, et qu'il nous a semblé judicieux d'analyser. Cette variante tente de reproduire l'algorithme 3 du papier sans utiliser les primitives de réduction (en l'occurrence, `reduceByKey`) appropriées de Spark. Après le `groupByKey`, au lieu de chercher simplement le minimum, le programme force un tri complet de la liste des voisins en mémoire Python via `sorted(list(values))` pour en extraire le premier élément.

```

1 sorted_vals = sorted(list(values))
2 min_val = sorted_vals[0] # simule values.next()

```

Cette méthode constitue un véritable faux-pas. Le *secondary sort* originel de MapReduce consiste à laisser le framework trier les données avant leur arrivée en RAM. Ici, le tri s'effectue

1. Lien GitHub : <https://github.com/louis-monier/SparkProject/blob/master/ccf-project-pyspark.ipynb>

dans la RAM. On cumule ainsi le goulot d'étranglement spatial $\mathcal{O}(N)$ de la variante 1 avec une complexité temporelle lourde $\mathcal{O}(N \log N)$ imposée au CPU pour le tri. C'est finalement une opération qui s'avère encore plus lourde que l'algorithme vanilla !

Ayant exploré les pistes naïves, on peut désormais se focaliser sur l'implémentation de versions *optimisées* adaptables en PySpark. En effet, on le rappelle à nouveau ici, l'implémentation décrite dans l'article original est destinée à tourner sur un cluster Hadoop, qui dispose nativement d'un mécanisme de *secondary sort*. Pour reproduire ce comportement sous Spark, il faut adopter une stratégie fondée sur un tri local des données au moment du shuffle et l'utilisation de fonctions natives (qu'on appelle également des *primitives*). L'objectif est de transformer l'étape de réduction en un **algorithme en ligne**, c'est-à-dire un traitement séquentiel du flux de données sans matérialisation complète des voisins.

3.2.3 Variante 3 : Optimisation Spark par réduction et jointure

Pour s'affranchir de la limite de RAM imposée par `groupByKey`, nous avons repensé l'algorithme en utilisant des fonctions d'agrégation de base.

Recherche des minima locaux

Au lieu de regrouper les voisins, on utilise `reduceByKey(min)`. Cette opération calcule le plus petit voisin localement sur chaque partition avant de l'envoyer sur le réseau. Le résultat est un RDD très léger associant chaque nœud à son minimum absolu. Pour réduire encore la taille de ce RDD, et ne propager le minimum que s'il est inférieur à l'identifiant du sommet lui-même, on filtre les paires (c'est le contenu de `emit_1`).

Propagation par jointure

Pour propager ce minimum aux autres voisins sans jamais créer de liste en mémoire, on passe par une étape de jointure (via `join`) entre les arêtes d'origine (u, v) et les minima validés (u, m) . Le *reducer* reçoit le flux $(u, (v, m))$ et émet (v, m) à la volée.

Enfin, on itère sur les résultats de la jointure, pour lier le voisin v au minimum m . Si $v = m$, aucune paire n'est émise, sinon, (v, m) est ajoutée à `emit_2`.

Algorithm 3 CCF-Iterate Optimised Primitive (v1)**Input** : RDD des arêtes (u, v) , accumulateur Spark (variable globale)**Output** : RDD des nouvelles paires

```

1 def ccf_iterate_optimised(rdd, new_pair_accum):
2
3     num_parts = rdd.getNumPartitions()
4     bothDirections = rdd.flatMap(lambda x: [(x[0], x[1]), (x[1], x[0])]) \
5                             .partitionBy(num_parts) \
6                             .persist()
7
8
9     mins = bothDirections.reduceByKey(min, numPartitions=num_parts)
10    mins_filtered = mins.filter(lambda x: x[1] < x[0])
11    emit_1 = mins_filtered
12
13
14    # La jointure se fait sur la key
15    # Join result : (key, (v, min_val))
16    joined = bothDirections.join(mins_filtered, numPartitions=num_parts)
17
18    def process_joined(item):
19        key, (v, min_val) = item
20        if v != min_val:
21            new_pair_accum.add(1)
22            return [(v, min_val)]
23        return []
24
25    emit_2 = joined.flatMap(process_joined)
26    result = emit_1.union(emit_2)
27
28    return result

```

Un avant goût des défis rencontrés

Contrairement aux approches précédentes, cette méthode utilise le RDD initial `bothDirections` à deux reprises : une fois pour calculer les minima, et une seconde fois pour effectuer la jointure. Dans le contexte de la lazy evaluation de Spark, cette double dépendance crée une ramification (concrètement, une sorte de losange) dans le plan d'exécution. Lors d'itérations successives, ce plan croît de manière exponentielle. Il est donc impératif de forcer une coupure de la lignée des données via la méthode `localCheckpoint()` à chaque fin de boucle, sous peine de saturer le gestionnaire de tâches de Spark.

Par ailleurs, cette approche résout le problème de la limite de RAM en respectant l'esprit du *secondary sort* (mémoire spatiale $\mathcal{O}(1)$ par nœud). En déléguant le calcul du minimum aux *workers* via `reduceByKey` (et donc un calcul des minima de manière locale), elle garantit la scalabilité de l'algorithme face à des graphes de grande dimension. Cependant, cette robustesse matérielle a un coût logiciel : l'utilisation couplée de `reduceByKey` et de `join` multiplie les transferts de données sur le réseau. Dans le contexte spécifique de PySpark, chaque *shuffle* impose une lourde pénalité due à la sérialisation et la désérialisation des données (multiples processus entre la JVM et Python). Ainsi, bien qu'en théorie adaptée pour une mise à l'échelle, cette méthode se révèle paradoxalement beaucoup plus lente que l'approche *vanilla* sur des graphes de petite ou moyenne taille, un comportement contre-intuitif sur lequel nous reviendrons au moment de l'analyse expérimentale.

3.2.4 Variante 4 : Optimisation avancée par Secondary Sort natif

Pour reproduire au plus près le comportement de Hadoop décrit dans l'article, cette dernière itération utilise une clé composite et la transformation `repartitionAndSortWithinPartitions`. L'objectif est de transformer l'étape de réduction en un traitement séquentiel strict du flux de données.

Phase Map

Contrairement aux versions précédentes, le couple (u, v) devient directement la **clé** et la valeur devient `None`. Ce choix est fondamental : la clé composite permet d'imposer un ordre lexicographique naturel lors du tri, d'abord sur u , puis sur v .

Phase Shuffle et Tri Local

L'appel à `repartitionAndSortWithinPartitions` combine deux opérations massivement optimisées en C/Scala sous le capot de Spark :

- **Repartition** : Nous redéfinissons la fonction de hachage pour qu'elle ne cible *que* le premier élément u . Ainsi, toutes les paires ayant le même u aboutissent physiquement sur la même partition. Cette étape est cruciale : si le partitionnement est effectué sur la clé complète (u, v) , alors les couples associés à un même u sont dispersés entre plusieurs partitions. Les comparaisons ne peuvent alors plus être réalisées localement et les données se retrouvent réparties dans différentes partitions, ce qui empêche un traitement séquentiel correct.
- **SortWithinPartitions** : Au sein de la partition, Spark trie les clés composites. Pour un même u , les voisins v apparaissent donc dans un ordre strictement croissant.

Phase Reduce (algorithme en ligne)

Les données arrivent triées sous la forme d'un flux continu : $(u_1, v_1), (u_1, v_2), (u_2, w_1), \dots$. Le traitement (via `mapPartitions`) consiste à parcourir séquentiellement ce flux. Lorsqu'un nouveau nœud u apparaît dans le flux, son tout premier voisin observé v_1 est mathématiquement garanti d'être le minimum. L'algorithme fonctionne donc à la volée (en *streaming*).

Le flux est parcouru une seule fois, l'empreinte mémoire reste constante $\mathcal{O}(1)$ par clé, et le nombre de *Shuffles* est réduit à un seul. Cette implémentation représente l'état de l'art pour cet algorithme sous Spark.

Algorithm 4 CCF-Iterate Optimised (Secondary Sort) (v2)**Input** : RDD des aretes (u, v) , accumulateur Spark**Output** : RDD des nouvelles paires

```

1 def ccf_iterate_secondary_sort(rdd_edges, new_pair_accum):
2     bothDirections = rdd_edges.flatMap(
3         lambda e: [((e[0], e[1]), None), ((e[1], e[0]), None)]
4     )
5
6     # Partition sur u uniquement
7     def part_by_u(key):
8         u, v = key
9         return portable_hash(u)
10
11    # Shuffle + tri local
12    sorted_pairs = bothDirections.repartitionAndSortWithinPartitions(
13        numPartitions=bothDirections.getNumPartitions(),
14        partitionFunc=part_by_u
15    )
16
17    # Scan sequentiel dans chaque partition
18    def reduce_partition(it):
19        cur_u = None
20        min_v = None
21        ignore = False
22
23        for (u, v), _ in it:
24
25            if u != cur_u:
26                cur_u = u
27                min_v = v
28
29            if cur_u <= min_v:
30                ignore = True
31            else:
32                ignore = False
33                new_pair_accum.add(1)
34                yield (cur_u, min_v)
35
36        else:
37            if ignore:
38                continue
39            if v != min_v:
40                new_pair_accum.add(1)
41                yield (v, min_v)
42
43    return sorted_pairs.mapPartitions(reduce_partition)

```

3.3 La boucle principale

La boucle principale est le cœur de l'orchestration. C'est ici que s'enchaînent les phases CCF-Iterate et CCF-Dedup jusqu'à convergence totale. Au-delà de la simple logique algorithmique itérative, cette boucle intègre des mécanismes d'optimisation strictement nécessaires pour garantir la stabilité et les performances sous PySpark. Bien souvent, retirer ou modifier ces lignes aboutissaient en l'échec de l'exécution de l'algorithme. Le suspect majeur de ces échecs étant, selon nous, les variations et incohérences des systèmes d'exploitation : par exemple, sous Windows 11, lancer les versions optimisées amenaient à des conflits d'écriture dans les dossiers locaux de Spark, problèmes que nous n'avons plus rencontrés en lançant les benchmarks sous Linux (Ubuntu 24.04 LTS).

Suivi de la convergence

Conformément au papier, l'algorithme doit s'arrêter lorsqu'aucune nouvelle paire n'est générée lors de la phase d'itération, signe que toutes les composantes connexes sont stabilisées sur leur identifiant minimum. Pour agréger cette information dans un environnement distribué, nous utilisons un **accumulateur Spark** (`sc.accumulator(0)`). Partagé entre tous les workers, il permet d'incrémenter le compteur global de nouvelles paires de manière concurrente.

Déclenchement de la lazy evaluation

Comme mentionné précédemment, si aucune action n'est appelée par le programme, la transformation n'est jamais physiquement exécutée, l'accumulateur reste à zéro, et la boucle s'arrête prématurément dès la première itération. L'appel à une action explicite, en l'occurrence `rdd.count()`, est donc indispensable pour déclencher les calculs et mettre à jour la valeur de la condition d'arrêt.

Troncature du graphe d'exécution via checkpointing

À chaque itération, Spark ajoute les nouvelles transformations au plan d'exécution. Ce graphe croît de manière exponentielle, le temps nécessaire pour simplement planifier les tâches finit par dépasser largement le temps de calcul effectif. L'utilisation de `rdd.localCheckpoint()` permet de matérialiser le RDD et d'arrêter l'expansion de cet arbre. À chaque itération, Spark repart ainsi d'un plan d'exécution court et optimisé.

Gestion dynamique de la RAM

Pour éviter une saturation progressive de la RAM, il est crucial de libérer les ressources au fur et à mesure que l'algorithme s'exécute. `cached_rdd.unpersist()` indique au gestionnaire de mémoire que le RDD intermédiaire de l'itération précédente n'est plus utile, libérant ainsi la RAM pour le calcul de l'itération courante.

Ci-dessous, les éléments principaux de la boucle itérative de l'algorithme CCF :

```

1 def ccf(sc, rdd, method="vanilla", customDedup=True):
2
3     iteration = 1
4
5     while True:
6         new_pair_accum = sc.accumulator(0)
7
8         if method == "vanilla":
9             iterated_rdd, cached_rdd = ccf_iterate_vanilla(rdd,
10 new_pair_accum)
11         elif method == "sec_sort_naive":
12             iterated_rdd, cached_rdd = ccf_iterate_SecondSort_naive(rdd,
13 new_pair_accum)
14         elif method == "optimised":
15             iterated_rdd, cached_rdd = ccf_iterate_secondary_sort(rdd,
16 new_pair_accum)
17         else:
18             raise ValueError("Unknown method. Please choose in [vanilla,
19 sec_sort_naive, optimised]")
20
21         rdd = ccf_dedup(iterated_rdd) if customDedup else iterated_rdd.
22 distinct()
23
24         count_elements = rdd.count()
25
26         if cached_rdd is not None:
27             cached_rdd.unpersist()
28
29         new_pairs = new_pair_accum.value
30         print(f"[{method}] Iteration {iteration}: {new_pairs} new pairs.")
31
32         if new_pairs == 0:
33             break
34
35         iteration += 1
36
37     return rdd, iteration

```

Chapitre 4

Analyse expérimentale et discussions

L'étude expérimentale s'appuie sur un environnement contrôlé et des jeux de données variés afin d'évaluer le comportement et la scalabilité des différentes variantes de l'algorithme CCF. L'analyse porte à la fois sur des graphes synthétiques générés selon des modèles réalistes et sur des graphes réels issus de réseaux complexes, permettant ainsi d'examiner les performances, les itérations nécessaires et les contraintes liées aux architectures distribuées.

4.1 Environnement expérimental

Les expérimentations ont été menées dans un environnement contrôlé permettant de reproduire, à l'échelle d'une machine unique, les principales contraintes d'exécution d'un cluster distribué.

Configuration du Spark context

- Le gestionnaire de cluster a été configuré en mode pseudo-distribué via l'instruction `local[4]`, allouant 4 cœurs logiques (représentant 4 *workers*) pour l'exécution des tâches.
- La mémoire du driver a été fixée à 16 Go (`spark.driver.memory`) et celle des exécuteurs à 4 Go (`spark.executor.memory`).
- Le `localCheckpointing` a été activé sur le système de fichiers local afin de tronquer le graphe d'exécution et de prévenir les overflows lors des itérations successives.

Configuration matérielle

- Processeur : Intel Core i7-1165G7 (11^e génération), 4 cœurs physiques, 8 threads, fréquence maximale 4.7 GHz, cache L3 de 12 Mo.
- Mémoire : 16 Go DDR4 3200 MHz (2 × 8 Go, configuration dual channel).
- Stockage : SSD NVMe Micron 2210 (512 Go).
- Système d'exploitation : Linux 64 bits.

Cette configuration permet d'étudier le comportement de l'algorithme dans un contexte où la mémoire principale est suffisante pour limiter le swapping disque, tout en conservant les coûts liés aux opérations de shuffle et de sérialisation propres au modèle Spark. L'utilisation

d'un SSD NVMe réduit également l'impact des écritures intermédiaires et du *checkpointing* local.

Les conditions expérimentales diffèrent significativement de celles décrites dans l'article original, qui reposait sur un cluster Hadoop composé de 80 nœuds avec des écritures disque distribuées. Les résultats obtenus ici doivent donc être interprétés dans le cadre d'une exécution mono-machine simulant un environnement distribué.

4.2 Jeux de données

Afin d'évaluer le comportement des algorithmes sur des graphes de tailles et de structures variées, plusieurs types de jeux de données ont été utilisés. L'objectif est d'analyser à la fois la scalabilité sur des graphes synthétiques et le comportement sur des graphes réels issus de réseaux complexes.

4.2.1 Graphes synthétiques

Les graphes synthétiques ont été choisis pour offrir un cadre contrôlé permettant d'évaluer la performance et la scalabilité de l'algorithme CCF, tout en reproduisant certaines propriétés structurales typiques des réseaux réels.

Graphes de type Small-World

Des graphes synthétiques de tailles croissantes ont été générés à l'aide de la bibliothèque NetworkX, en utilisant le modèle de Barabási–Albert. En effet, en générant des graphes parfaitement au hasard, on risquerait de perdre l'interprétabilité des temps d'exécution et de l'analyse expérimentale : il faut donc que l'aspect synthétique du graphe copie un tant soit peu les données réelles. Et justement, ce modèle permet de produire des graphes présentant une distribution en loi de puissance des degrés, caractéristique de nombreux réseaux réels (réseaux sociaux, web, etc.).

Trois tailles ont été considérées :

- **SmallWorld_1000** : 1 000 sommets, environ 5 000 arêtes ;
- **SmallWorld_10k** : 10 000 sommets, environ 50 000 arêtes ;
- **SmallWorld_100k** : 100 000 sommets, environ 500 000 arêtes.

Ces graphes permettent d'évaluer la montée en charge progressive de l'algorithme dans un cadre contrôlé, tout en conservant une structure réaliste.

Graphes linéaires (cas pathologique)

Afin d'analyser le comportement de l'algorithme dans le seul scénario défavorable mentionné dans le papier, nous avons de plus généré un graphe linéaire. Un graphe linéaire de taille n est constitué d'une chaîne de sommets où chaque sommet est relié uniquement à son successeur (c'est un *simple path* au sens employé en théorie des graphes).

Dans ce type de structure, le diamètre du graphe est maximal ($d = n - 1$). Or, le nombre d'itérations nécessaires pour la propagation complète de l'identifiant minimal dépend directement du diamètre. Ce cas représente donc une situation proche du pire cas théorique pour l'algorithme.

Le jeu de données suivant a été utilisé :

- **LineGraph_1000** : 1 000 sommets organisés en chaîne.

4.2.2 Graphes réels

En complément des graphes synthétiques, plusieurs jeux de données réels issus de réseaux complexes ont été utilisés :

- **Web-Google** ([Stanford Network Analysis Project \(SNAP\) \(2002b\)](#)) : graphe du réseau web Google (875 713 sommets et 5 105 039 arêtes) ;
- **Web-BerkStan** ([Stanford Network Analysis Project \(SNAP\) \(2002a\)](#)) : graphe du web universitaire (685 230 sommets et 7 600 595 arêtes) ;
- **LiveJournal** ([Stanford Network Analysis Project \(SNAP\) \(2007\)](#)) : réseau social en ligne (4 847 571 sommets et 68 993 773 arêtes) ;
- **Orkut** ([Stanford Network Analysis Project \(SNAP\) \(2012\)](#)) : réseau social de grande taille (3 072 441 sommets et 117 185 083 arêtes).

Ces graphes présentent des structures hétérogènes, avec des distributions de degrés fortement asymétriques et de grandes composantes connexes. Ils permettent d'évaluer le comportement des implémentations dans des conditions proches d'applications réelles à grande échelle.

L'ensemble de ces jeux de données permet ainsi d'étudier :

- l'évolution du temps d'exécution en fonction de la taille du graphe ;
- l'impact de la topologie du graphe (diamètre, distribution des degrés) ;
- le comportement dans des cas favorables et pathologiques ;
- la scalabilité globale des différentes variantes de l'algorithme.

4.3 Résultats

Les expériences présentent les performances des différentes variantes de l'algorithme CCF sur plusieurs graphes synthétiques et réels. Pour chaque graphe, nous reportons le temps d'exécution total, le nombre d'itérations nécessaires à la convergence et le nombre de composantes connexes détectées, permettant ainsi d'évaluer à la fois l'efficacité et la robustesse des différentes implémentations.

Graph	Method	Time (s)	Iterations	Components
SmallWorld_1000	vanilla	2.50	4	1
	sec_sort_naive	2.30	4	1
	optimised	2.29	4	1
SmallWorld_10k	vanilla	2.88	4	1
	sec_sort_naive	2.90	4	1
	optimised	3.03	4	1
SmallWorld_100k	vanilla	13.02	5	1
	sec_sort_naive	12.98	5	1
	optimised	16.63	5	1
LineGraph_1000	vanilla	9.34	12	1
	sec_sort_naive	9.45	12	1
	optimised	11.45	12	1
Web-Google	vanilla	117.55	8	2746
	sec_sort_naive	118.73	8	2746
	optimised	187.93	8	2746
Web-Berkstan	vanilla	83.77	13	676
	sec_sort_naive	81.73	13	676
	optimised	125.16	13	676
LiveJournal	vanilla	1850.47	7	914
	sec_sort_naive	1574.91	7	914
	optimised	1404.87	7	914
Orkut	vanilla	4146.74	5	1
	sec_sort_naive	3759.21	5	1
	optimised	3749.99	5	1

Table 4.1 : Temps d'exécution, nombre d'itérations et nombre de composantes connexes pour chaque graphe et chaque méthode.

Les résultats mettent en évidence plusieurs tendances importantes.

Premièrement, le nombre d'itérations dépend principalement de la structure du graphe et non de la méthode utilisée. Par exemple, les graphes SmallWorld convergent rapidement (4 à 5 itérations), tandis que le graphe linéaire nécessite 12 itérations en raison de son diamètre élevé. Au demeurant, cela illustre parfaitement la théorie présentée par Kardes et al.

Deuxièmement, le nombre de composantes connexes détectées est identique pour les trois méthodes sur chaque graphe, ce qui confirme la correction fonctionnelle des implémentations.

Enfin, l'écart de performance révèle un point de bascule fondamental pour la scalabilité. Si pour les graphes de petite à moyenne taille (jusqu'à Web-Google), les différences de temps d'exécution entre les trois variantes restent relativement faibles et la méthode vanilla domine, sur les graphes massifs (LiveJournal et Orkut), la version optimisée avec secondary sort prend largement l'avantage, réduisant le temps d'exécution de plus de 400 secondes sur LiveJournal.

4.4 Discussion

Les résultats expérimentaux présentés précédemment révèlent non seulement l'efficacité des différentes variantes de l'algorithme CCF, mais mettent également en lumière les compromis architecturaux et les contraintes matérielles rencontrées lors de l'exécution sous PySpark.

4.4.1 Analyse des performances et des architectures

L'analyse des expérimentations met en lumière un compromis fondamental en calcul distribué : l'équilibre entre complexité spatiale et complexité de communication (coût du shuffle sur le réseau). La mise en application de l'algorithme CCF sous PySpark nous a confrontés à des défis de bas niveau qui ont forgé nos choix au fur et à mesure de l'implémentation.

Retour sur l'échec de la première optimisation

Notre première tentative pour contourner la limite de mémoire imposée par le `groupByKey` consistait à utiliser des primitives de réduction (ici, `reduceByKey`) suivies d'un `join`. Cette méthode n'a pas été retenue dans notre comparatif final en raison de performances médiocres en pratique, quand bien même elle fût en accord avec l'effort de complexité spatiale. Cet échec s'explique par l'architecture même de PySpark et ses échanges avec la machine virtuelle Java (JVM). L'utilisation couplée d'une réduction et d'une jointure force la création de multiples shuffles par itération. Or, lors de chacun d'entre eux, PySpark doit sérialiser les données Python en bytecode Java, les transférer sur le réseau, puis les désérialiser en objets Python pour l'étape suivante (c'est le «pickling»). De reste, cette approche créait une dépendance récursive dans le graphe d'exécution, provoquant des temps de planification bien trop élevés malgré l'utilisation du checkpointing.

Version optimisée retenue sur les petits et moyens graphes

La version optimisée présentée dans les résultats repose sur un unique shuffle via la fonction bas niveau `repartitionAndSortWithinPartitions`. Pourtant, sur des graphes de taille faible à moyenne (comme Web-Google), les méthodes vanilla et `sec_sort_naive` se révèlent paradoxalement plus rapides (117s pour Vanilla contre 187s pour l'optimisée). Ce résultat contre-intuitif s'explique par le surcoût de l'ordonnancement distribué. La méthode vanilla n'effectue qu'un simple shuffle de regroupement. Une fois les données regroupées, l'intégralité du calcul (recherche du minimum sur une liste) est exécutée dans la RAM locale de l'interpréteur Python. La méthode optimisée, en revanche, demande à Spark de calculer des hachages complexes, de réorganiser physiquement les partitions, et d'appliquer un tri natif avant même de confier les données à Python.

Mise à l'échelle sur les graphes massifs

La dynamique s'inverse à grande échelle. Sur des réseaux de très grande taille comme Live-Journal (69 millions d'arêtes), les limites physiques de la méthode vanilla apparaissent (temps d'exécution avoisinant les 1850 secondes), tandis que la méthode optimisée démontre sa supériorité architecturale (1404 secondes). Dans chaque graphe, le traitement des hubs (sommets comptant un grand nombre de voisins) contraint la méthode vanilla à allouer dynamiquement un tableau contenant des millions d'entiers dans la RAM d'un seul *worker*. Cela sature le processeur à cause du déclenchement continu du garbage collector (GC) pour gérer la mémoire. À l'inverse, l'approche `repartitionAndSortWithinPartitions`, qui permet un traitement des données sous forme d'un flux préalablement trié, se montre plus efficace. Le *reducer* n'a besoin de mémoriser qu'une seule variable à la fois (complexité spatiale $\mathcal{O}(1)$). La charge est distribuée efficacement. Ainsi, bien qu'elle impose un lourd coût d'initialisation sur les petits jeux de données, cette méthode est la seule véritablement scalable pour traiter des graphes dont la densité excède les capacités de la RAM.

4.4.2 Points forts et limites du cadre CCF

Au-delà des performances brutes, il est essentiel d'évaluer le cadre CCF sous l'angle de ses propriétés intrinsèques et de ses contraintes opérationnelles. Cette analyse met en évidence ce que l'algorithme apporte de robuste et efficace, ainsi que les situations où ses limites deviennent significatives, afin de mieux situer son applicabilité dans différents contextes de graphes et d'architectures distribués.

Points forts

- **Garantie de convergence** : D'un point de vue mathématique, la stratégie de propagation du *Min-ID* est déterministe. Elle garantit une convergence absolue, indépendamment de la distribution initiale des arêtes sur le cluster ou de l'ordre d'exécution des tâches.
- **Mise à l'échelle spatiale ($\mathcal{O}(1)$)** : Lorsqu'il est correctement implémenté avec *secondary sort*, le framework est entièrement indépendant de la limite de RAM des *workers*. Il permet de traiter des réseaux abritant des hubs sans erreur de mémoire.
- **L'algorithme ne requiert aucune vision globale du graphe**. Le calcul repose exclusivement sur des échanges locaux de voisinage, ce qui s'aligne parfaitement avec le paradigme MapReduce.

Limites

- **Sensibilité à la topologie** : Comme l'a prouvé notre expérimentation sur le graphe linéaire, le nombre d'itérations de CCF est directement conditionné par le diamètre de la plus grande composante (pire cas en $d+1$ itérations). L'algorithme est donc sous-optimal pour des réseaux topologiquement très étirés.
- **Coût de nettoyage des redondances** : La phase CCF-Iterate génère de façon exponentielle de nouvelles paires, dont une majorité est redondante. L'obligation d'appliquer CCF-Dedup à chaque boucle impose des opérations de shuffle, dont le contre-coup est accentué par la sérialisation en PySpark.
- **Hadoop vs Spark** : Le framework CCF a été pensé pour exploiter le tri natif sur disque de Hadoop. Sa transposition à Spark n'est pas immédiate : elle requiert un effort particulier sur les éléments de bas niveau, dont font partie le checkpointing, la mise en cache et le partitionnement explicite. En un certain sens, cela traduit également le fait que les optimisations via MapReduce ne sont pas universellement applicables.

Chapitre 5

Conclusion

Dans ce projet, nous avons implémenté et comparé les variantes de l'algorithme **CCF** proposé par ([Kardes et al. \(2014\)](#)), en l'adaptant au modèle RDD de Spark.

Ne disposant pas d'un cluster distribué, nos expérimentations ont été réalisées sur un seul ordinateur. Nous n'avons donc pas pu reproduire les tests sur graphes massifs présentés dans l'article, et notre analyse s'est limitée à des graphes de taille modérée.

Les résultats expérimentaux montrent que les différences de temps d'exécution entre les implémentations restent globalement faibles. Sur les petits graphes, la version *vanilla* (basée sur `groupByKey`) est légèrement plus rapide. En revanche, lorsque la taille du graphe augmente, la tendance s'inverse : l'implémentation reposant sur le *secondary sort* devient légèrement plus performante.

Cette évolution est cohérente avec l'esprit du papier. La version *vanilla* nécessite de matérialiser les listes d'adjacence en mémoire, ce qui implique de stocker localement les voisins d'un sommet dans la RAM du worker. À l'inverse, la version fondée sur le *secondary sort* procède par traitement en flux, sans construire explicitement ces listes, garantissant ainsi une empreinte mémoire constante par clé.

Ainsi, même si les écarts mesurés restent modérés sur nos jeux de données, nos observations confirment l'intuition centrale de l'article : les optimisations liées au tri et au contrôle du shuffle prennent davantage de sens lorsque la taille du graphe augmente. À performances comparables, l'implémentation basée sur le *secondary sort* constitue donc un choix plus robuste dans une perspective de passage à l'échelle.

Bibliographie

- Dean, J. and Ghemawat, S. (2004), Mapreduce : Simplified data processing on large clusters, *in* 'OSDI'04 : Sixth Symposium on Operating System Design and Implementation', San Francisco, CA, pp. 137–150.
- Kardes, H., Agrawal, S., Wang, X. and Sun, A. (2014), Ccf : Fast and scalable connected component computation in mapreduce, *in* '2014 International Conference on Computing, Networking and Communications (ICNC)', pp. 994–998.
- Stanford Network Analysis Project (SNAP) (2002a), 'web-BerkStan : Berkeley–stanford web graph (snap dataset)', <https://snap.stanford.edu/data/web-BerkStan.html>.
- Stanford Network Analysis Project (SNAP) (2002b), 'web-Google : Web graph from google programming contest (snap dataset)', <https://snap.stanford.edu/data/web-Google.html>.
- Stanford Network Analysis Project (SNAP) (2007), 'soc-LiveJournal1 : Livejournal social network (snap dataset)', <https://snap.stanford.edu/data/soc-LiveJournal1.html>.
- Stanford Network Analysis Project (SNAP) (2012), 'com-Orkut : Orkut social network and ground-truth communities (snap dataset)', <https://snap.stanford.edu/data/com-Orkut.html>.
- Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauly, M., Franklin, M. J., Shenker, S. and Stoica, I. (2012), Resilient distributed datasets : A Fault-Tolerant abstraction for In-Memory cluster computing, *in* '9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)', USENIX Association, pp. 15–28.
- Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S. and Stoica, I. (2016), 'Apache spark : a unified engine for big data processing', *Commun. ACM* **59**(11), 56–65.

Annexe A

Code source

Toutes les composantes du projet, à savoir :

- Implémentation des algorithmes CCF
- Génération de graphes
- Exécution du benchmark
- Graphes utilisés dans la partie expérimentale
- Bibliothèques requises

se trouvent sur le dépôt GitHub (https://github.com/Apayao/BigData_CCF).